In-Depth Study Notes: Oracle SQL Null Handling, Conditional Branching, and Type Conversion

As an expert Oracle SQL Developer, mastering data sanitization and dynamic conditional logic is essential for building resilient enterprise reports. Based on the execution logs provided in the source files (CH4-1.TXT and CH4-2.TXT), these notes break down the architecture of **General Null Functions**, **Conditional Branching (DECODE / CASE)**, and **String-to-Number Conversion (TO_NUMBER)**.

Concept 1: Advanced Null Handling (NVL2, NULLIF, COALESCE)

Core Purpose:NULL represents an unknown or missing value. Because mathematical operations with a NULL automatically return NULL 1, 2, databases require intercept functions to dynamically swap missing values with hardcoded defaults to prevent calculations from collapsing 3, 4.

Keyword Breakdown:

- **NVL(expr1, expr2)**: If expr1 is null, returns expr2. If expr1 is not null, returns expr1 3.
- **NVL2(expr1, if_not_null, if_null)**: If expr1 has data, it replaces it with the second argument. If it is null, it replaces it with the third argument 5.
- **NULLIF(expr1, expr2)**: Compares two arguments. If they are exactly identical, it forces a NULL return. If they differ, it returns the first argument 6. Highly useful for preventing "Divide by Zero" errors.
- **COALESCE(expr1, expr2, ... expr_n)**: Accepts infinite arguments. It scans left-to-right and returns the **very first non-null value** it encounters 7.

Comparison of Null Handlers:

Function,Argument Limit,Behavior Focus,Strict Data Type Matching?
NVL,Exactly 2,Intercepts only missing data.,"Yes, if arg1 is numeric/date 3."
NVL2,Exactly 3,Overwrites both present AND missing data.,Arg2 and Arg3 must match 5.
NULLIF,Exactly 2,Creates a NULL from matching data.,Both args must match data type 8.
COALESCE,Infinite,Fallback prioritization list.,All arguments must match 7.

Syntax Breakdown:

```
SELECT COALESCE(col_1, col_2, 'default_value'),
       NVL2(target_col, 'Has Data', 'Missing Data')
FROM table_name;
```

Detailed Example: An HR schema has employees who either receive a commission or do not. The business requires paying 50% extra to those currently receiving a commission, but guaranteeing a baseline 12% commission to those who currently have none (NULL).

```
SELECT COMMISSION_PCT,
       NVL2(COMMISSION_PCT, COMMISSION_PCT * 1.5, 0.12) AS
NEW_COMMISSION
FROM EMPLOYEES;
```

Execution: Dynamically scales existing commissions or intercepts NULLs and injects 0.12 2, 9.

Execution Logic: These functions operate during the row-by-row projection phase. For COALESCE, the engine utilizes **short-circuit evaluation**. It evaluates argument 1; if it is not

null, it projects the value and completely skips evaluating the remaining arguments, saving CPU cycles 7.

Pro-Tips/Best Practices:

- **The ORA-01722 Data Type Trap:** If NVL(COMMISSION_PCT, 'NA') is executed, the query crashes 4. COMMISSION_PCT is a number, and 'NA' is a string. You must use implicit or explicit conversion to align them: NVL(TO_CHAR(COMMISSION_PCT), 'NA') 10, 11.

Concept 2: Conditional Branching (DECODE vs CASE)

Core Purpose: Standard SQL is declarative, not procedural. However, business rules often require IF-THEN-ELSE logic (e.g., "If Dept is 20, give a $500 raise; if Dept is 50, give a $1000 raise"). DECODE and CASE allow developers to embed complex branching logic directly inside the SELECT statement.

Keyword Breakdown:

- **DECODE(col, search, result, default)**: An older, **Oracle-specific** function. It only performs strict equality (=) checks 12.
- **CASE WHEN ... THEN (Searched Case)**: The ANSI-standard branching method. It supports complex inequalities (>, <), BETWEEN, IN, and LIKE operators 13, 14.
- **ELSE**: The fallback bucket if no WHEN conditions are met 13, 15.

Comparison of Branching Logic:

Feature,DECODE Function,CASE Statement
Vendor Support,Oracle exclusively.,ANSI Standard (Works on all RDBMS).
Operator Support,Strict Equality (=) only 12.,"Inequalities (>), BETWEEN, IN 13."
Readability,Difficult in deeply nested scenarios.,Highly structured and readable.

Syntax Breakdown (CASE):

```
SELECT column_name,
       CASE
           WHEN condition_1 THEN result_1
           WHEN condition_2 THEN result_2
           ELSE default_result
       END AS Alias
FROM table_name;
```

Detailed Example: The company declares a dynamic salary increment based on hiring eras: Hired before 1991 gets +10%, 1991-1994 gets +20%, 1995-1997 gets +30%, and anyone else gets +40% 16.

```
SELECT FIRST_NAME, HIRE_DATE,
       CASE
           WHEN HIRE_DATE < '01-JAN-91' THEN SALARY * 1.1
           WHEN HIRE_DATE BETWEEN '01-JAN-91' AND '31-DEC-94' THEN
SALARY * 1.2
           WHEN HIRE_DATE BETWEEN '01-JAN-95' AND '31-DEC-97' THEN
SALARY * 1.3
           ELSE SALARY * 1.4
       END AS NEW_SALARY
```

```
FROM EMPLOYEES;
```
Execution: Correctly processes the inequalities and applies the proper mathematical multiplier 17.

Execution Logic:CASE statements also use **short-circuit evaluation**. The engine evaluates the WHEN clauses sequentially from top to bottom. The millisecond a condition evaluates to TRUE, the engine applies the corresponding THEN logic and immediately exits the END block 13, ignoring all subsequent conditions.

Pro-Tips/Best Practices:

- **Always include an ELSE:** If an ELSE is omitted and a row fails to match any WHEN criteria, the CASE statement will silently return a NULL. Always define an ELSE (even if it's just ELSE SALARY) to prevent accidental data loss 15, 18.

Concept 3: String Conversion and Formatting (TO_NUMBER)

Core Purpose: When flat files or user interfaces pass numeric data into the database, it often arrives as a formatted string (e.g., '$1,234.56' or '1234.56-'). The CPU's Arithmetic Logic Unit (ALU) cannot perform addition or subtraction on a string containing a dollar sign 19. TO_NUMBER translates these formatted text literals into raw, computable numbers.

Keyword Breakdown & Format Masks:

- **TO_NUMBER(string, 'format_mask')**: Instructs the engine to strip away the defined symbols and return a clean integer or float 19.
- **9**: Represents a numeric digit (suppresses leading zeros) 19.
- **0**: Represents a numeric digit (forces leading zeros to display) 20.
- **G (Group) / , (Comma)**: Represents the thousands separator 21, 22.
- **D (Decimal) / . (Period)**: Represents the decimal point 22.
- **L (Local) / $**: Denotes the local currency symbol (e.g., $, Rs.) 23, 24.
- **S (Sign)**: Indicates the location of a positive/negative sign, like - or + 19.

Comparison of Formatting Output:

Raw Input String,Mask Required to Parse,Computed Value,Business Use Case
"'1,234'","'9,999' or '9G999' 21, 22",1234,Standard financial import.
'1234.56-','9999.99S' 19,-1234.56,Negative accounting ledgers.
'$1234','$9999' or 'L9999' 23,1234,UI string sanitization.
'Rs.1234','L9999' 24,1234,Localized currency import.

Syntax Breakdown:

```
SELECT TO_NUMBER('string_value', 'format_mask') + arithmetic_modifier
FROM DUAL;
```

Detailed Example: An application passes a localized Indian Rupee value 'Rs.1234' as a string, and the database needs to add it to a base ledger value of 10000. We must alter the session to recognize 'Rs.' as the local currency, then convert it 24.

```
ALTER SESSION SET NLS_CURRENCY = 'Rs.';

SELECT 10000 + TO_NUMBER('Rs.1234', 'L9999') AS LEDGER_TOTAL
FROM DUAL;
```

Execution: The database safely parses the string, strips the 'Rs.', and mathematically outputs 11234 24.

Execution Logic: When parsing letters like G, D, L, or U, the compiler does not look at standard ASCII rules. Instead, it queries the **NLS_SESSION_PARAMETERS** (National Language Support) stored in the database registry 25-27. If NLS_CURRENCY is set to $, then L maps to $. If the user alters their session (ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.|' 28), the engine dynamically updates its parsing rules globally for that session 29.

Pro-Tips/Best Practices:

- **ORA-01722 and Strict Masks:** The format mask provided in the second argument must precisely match the location of the symbols in the string. Using TO_NUMBER('$ 1234', '9999$') will crash because the dollar sign is mathematically on the wrong side of the mask 30.
- **Use Global Identifiers (G/D):** Always prefer G and D over hardcoded commas , and periods .. European databases often swap these (using commas for decimals). G and D natively adapt to the server's region, making your code globally portable 22.